

Tag 9

Funktionen rufen Funktionen & das main()-Muster

Python-Grundlagen-Schulung

Was du heute lernst

1. Wie genau **wandert ein Wert** in eine Funktion hinein und wieder heraus?
2. Wie kann eine Funktion **eine andere Funktion** benutzen?
3. Was ist `main()` – und warum schreibt man das fast immer?
4. Viel Übung: eigene kleine Programme bauen

Teil 1

Wie ein Wert durch die Funktion reist

Erinnerung an Tag 8

Eine Funktion ist eine **Maschine**:

 etwas rein →  Verarbeitung →  etwas raus

Heute schauen wir uns **ganz genau** an, was dabei im Inneren passiert.

Ein Beispiel

```
def berechne_wert(zahl):  
    return zahl * 2  
  
ergebnis = berechne_wert(10)
```

Was passiert hier eigentlich Schritt für Schritt?
Schauen wir es uns wie einen Comic an →

Schritt 1: Der Aufruf

```
ergebnis = berechne_wert(10)
```

Python sagt: "Ich soll `berechne_wert` mit der Zahl **10** aufrufen."

↓ Python springt jetzt zur Funktion

Schritt 2: In der Funktion ankommen

```
def berechne_wert(zahl):
```

Die **10** wird jetzt der Variable `zahl` gegeben.

`zahl` ist wie eine **Schublade**, in die kurz die 10 reingelegt wird.

Schritt 3: Rechnen

```
return zahl * 2
```

Python rechnet: $10 * 2 = 20$

Und schickt die **20 zurück** dorthin, wo die Funktion aufgerufen wurde.

Schritt 4: Zurück im Hauptprogramm

```
ergebnis = berechne_wert(10)
```

Jetzt wird aus dieser Zeile:

```
ergebnis = 20
```

ergebnis speichert jetzt die 20.

Der ganze Weg auf einen Blick

10 wird übergeben



kommt in der Funktion als `zahl` an



wird zu **20** verarbeitet



kommt als **20** zurück

Ein wichtiger Trick

```
mein_wert = 10  
ergebnis = berechne_wert(mein_wert)
```

Der Name `mein_wert` draußen und der Name `zahl` drinnen
müssen nicht gleich heißen!

Python interessiert sich nur für den **Wert** (die 10),
nicht für den Namen der Variable.

Warum ist das nützlich?

Du kannst dieselbe Funktion mit **ganz unterschiedlichen** Variablen oder Zahlen füttern.

Die Funktion selbst muss sich dafür **nie ändern**.

Jetzt übst du das an 3 kleinen Aufgaben im Colab-Notebook. 

Teil 2

Funktionen rufen Funktionen

Das Problem: eine riesige Funktion

Stell dir vor, eine Funktion soll **alles auf einmal** können:

Steuer berechnen, Rabatt berechnen, alles zusammenzählen...

Das wird schnell **unübersichtlich** und **schwer zu finden**,
wenn mal ein Fehler drin ist.

Die Lösung: aufteilen!

Statt **einer riesigen** Maschine baust du
mehrere kleine, spezialisierte Maschinen.

Eine große Maschine kann dann die kleinen Maschinen benutzen!

Bild: der Vorarbeiter

Ein **Vorarbeiter** macht nicht alles selbst.

Er sagt: "Du machst die Steuer, du machst den Rabatt" –
und setzt am Ende alles zusammen.

Das nennt man Modularisierung.

Zwei kleine, einfache Funktionen

```
def ermittle_steuer(netto_betrag):  
    return netto_betrag * 0.19  
  
def ermittle_rabatt(netto_betrag):  
    if netto_betrag > 1000:  
        return netto_betrag * 0.10  
    return 0.0
```

Jede Funktion kann **genau eine** Sache. Nicht mehr.

Eine Funktion, die die anderen benutzt

```
def kalkuliere_gesamtpreis(netto_betrag):  
    steuer = ermittle_steuer(netto_betrag)  
    rabatt = ermittle_rabatt(netto_betrag)  
  
    return netto_betrag + steuer - rabatt
```

`kalkuliere_gesamtpreis` **ruft** die beiden anderen Funktionen auf –
so, wie du im Hauptprogramm auch eine Funktion aufrufst!

Schritt für Schritt: `kalkuliere_gesamtpreis(1200)`

1. `ermittle_steuer(1200)` wird aufgerufen → liefert **228.0** zurück
2. Das landet in der Variable `steuer`
3. `ermittle_rabatt(1200)` wird aufgerufen → liefert **120.0** zurück (weil $1200 > 1000$)
4. Das landet in der Variable `rabatt`
5. Am Ende: $1200 + 228.0 - 120.0 = 1308.0$

Warum ist das so gut?

- ✓ Jede kleine Funktion kannst du **einzel**n testen
- ✓ Der Name verrät sofort, **was** die Funktion tut
- ✓ Eine kleine Funktion kannst du **später woanders wiederverwenden**

Merksatz

**Eine Funktion darf ruhig
eine andere Funktion benutzen!**



Teil 3

Die main()-Funktion

Ein neues Problem

Bisher haben wir Variablen einfach frei
im Notebook stehen lassen.

Bei kleinen Übungen kein Problem.

Aber bei **größeren Programmen** wird das schnell unübersichtlich –
man weiß nicht mehr, was wo herkommt.

Die Lösung: eine Haupt-Schublade

Wir packen **den gesamten Ablauf** unseres Programms
in **eine einzige Funktion**.

Diese Funktion nennen wir immer **main**
(englisch für "Haupt-").

So sieht das aus

```
def main():  
    print("Das Hauptprogramm läuft...")  
  
if __name__ == "__main__":  
    main()
```

Das ist der **Standard**, den du in fast jedem
"richtigen" Python-Programm findest.

Die main-Funktion

```
def main():  
    print("Das Hauptprogramm läuft...")
```

Hier steht **alles**, was dein Programm eigentlich tun soll:
Fragen stellen, andere Funktionen aufrufen, Ergebnisse zeigen.

Die geheimnisvolle letzte Zeile

```
if __name__ == "__main__":  
    main()
```

Vereinfacht gesagt bedeutet das:

"Wenn diese Datei direkt gestartet wird, dann starte main()."

Für dich als Anfänger reicht es, diese zwei Zeilen
immer genau so ans Ende zu schreiben.

Merksatz

`main()` bündelt den Ablauf.

`if __name__ == "__main__":` startet ihn.

Schreib diese zwei Zeilen **immer** am Ende deines Programms.

Teil 4

Jetzt du: modulare Programme bauen

Was du jetzt machst

Du baust drei kleine Programm-Systeme.

Jedes besteht aus:

1. Ein oder zwei **kleinen Hilfsfunktionen**
2. Einer `main()` -Funktion, die die Hilfsfunktionen benutzt
3. Der Abschluss-Zeile `if __name__ == "__main__": main()`

Die drei Programme



Server-Raum-Monitor

Temperatur bewerten und Alarm-Text erzeugen



Onboarding-Assistent

E-Mail und Benutzername für neue Mitarbeiter erzeugen



Ping-Latenz-Tester

Mit einer `while`-Schleife immer wieder Latenz abfragen

Alle Aufgaben und Lösungen findest du im **Colab-Notebook**.

Ein Beispiel zum Warmwerden

```
def bewerte_temperatur(temp):  
    if temp > 30:  
        return "kritisch"  
    return "normal"  
  
def main():  
    temp = float(input("Temperatur in °C: "))  
    status = bewerte_temperatur(temp)  
    print(status)  
  
if __name__ == "__main__":  
    main()
```

Zum Schluss

Warum das Ganze?

Ein Vergleich

Ohne Struktur: Ein Fehler? Du musst das ganze Programm von oben bis unten durchsuchen. 🙄

Mit kleinen Funktionen + main():

Ein Fehler in der E-Mail-Erzeugung?

Du schaust **nur** in `generiere_email` nach. 🎯

Was du heute gelernt hast

- ✓ Ein Wert reist von der Aufrufstelle in die Funktion – und mit `return` wieder zurück
- ✓ Eine Funktion darf **andere Funktionen** benutzen (Modularisierung)
- ✓ `main()` bündelt den Ablauf deines Programms an einer Stelle

Morgen: der große Projekttag – dein eigenes Spiel! 🎉